Lembar Kerja Praktikum P07 – Exception Handling

Lembar kerja ini menggantikan laporan praktikum. Isi **sambil** mengerjakan setiap langkah di modul, lalu kumpulkan di akhir sesi sebagai `LK-P07-<NIM>.pdf` bersama zip kode Anda.

Petunjuk pengisian

- Jawab singkat dan jelas. Satu sampai tiga kalimat per pertanyaan sudah cukup.
- Tempel output program sebagai **teks**, bukan tangkapan layar.
- Di L2 dan L3, isi kolom **prediksi** sebelum menjalankan program. Prediksi yang salah tidak mengurangi nilai; yang dinilai adalah penjelasan selisihnya.
- Total nilai LK 50 poin. Bagian L0 wajib diisi, tanpa poin.

L0 – Identitas dan parameter (wajib)

Isian	Jawaban
Nama	Hafizh Habibullah
NIM	2511531002
Kelas	B
Tanggal praktikum	Senin, 9 November 2026
Versi Java (baris pertama `java -version`)	25.0.1
Saldo awal e-wallet (`NimParams`)	Rp250000
Stok awal KOPI-001	4
Stok awal RDG-002	1

L1 – Membaca crash dan menangani input (8 poin)

(a) Bedah stack trace `CrashDemo` skenario 2 dan 3 (4 poin). Contoh pengisian untuk skenario 1 ada di modul.

Yang dicari	Skenario 2	Skenario 3
Jenis exception	NumberFormatException	NullPointerException
Checked atau unchecked?	Unchecked	Unchecked
Pesan	For input string: "dua”	Cannot invoke "shop.model.Address.getCity()" because the return value of "shop.demo.CrashDemo$Customer.getAddress()" is null
Baris kode Anda yang pertama (file:baris, method)	CrashDemo.java:28, method readQuantit, yaitu return Integer.parseInt(text)	CrashDemo.java:32, method shippingCity, yaitu return customer.getAddress().getCity()
Dipanggil dari (file:baris, method)	CrashDemo.java:43, method Main	CrashDemo.java:44, method Main

Yang dicari	Skenario 2	Skenario 3
Apa penyebabnya, dengan kata-kata Anda sendiri?	Input yang diberikan adalah “dua”, sedangkan input yang diminta berjenis int, bukan String.	Objek address pada Customer belum diinisialisasi sehingga bernilai null, tetapi program mencoba memanggil method getCity() dari objek tersebut.

(b) Jumlah `-2` (2 poin). Sebelum `ShopApp` diperbaiki, apa yang terjadi saat Anda memilih produk 2 lalu mengetik jumlah `-2`? Mengapa tidak muncul `NumberFormatException`? Apa beda "format input salah" dengan "aturan bisnis dilanggar"?

Input -2 tidak menghasilkan NumberFormatException karena masih merupakan bilangan bulat yang valid dan dapat dikonversi oleh Integer.parseInt(). Masalahnya, nilai tersebut melanggar aturan bisnis karena jumlah pembelian tidak boleh negatif. Berbeda dengan input "x" yang menyebabkan NumberFormatException karena tidak dapat dikonversi menjadi integer. Oleh karena itu, diperlukan validasi tambahan untuk memastikan jumlah pembelian harus lebih dari 0.

Format input salah berarti data tidak sesuai dengan tipe data yang diharapkan, contohnya memasukkan "x" saat program meminta angka. Sedangkan aturan bisnis dilanggar berarti format input sudah benar, tetapi nilainya tidak sesuai ketentuan, contohnya memasukkan jumlah pembelian -2 karena jumlah tidak boleh negatif.

(c) `if` atau `catch`? (2 poin). Mengapa nomor produk di luar katalog dicegah dengan `if`, sedangkan teks yang bukan angka ditangkap dengan `catch (NumberFormatException _)`?

Teks yang bukan angka ditangani dengan catch karena dapat menyebabkan NumberFormatException saat dikonversi menggunakan Integer.parseInt(). Sementara itu, nomor produk di luar katalog dicegah dengan if karena input tersebut tetap berupa angka yang valid, tetapi tidak sesuai dengan rentang nomor produk yang tersedia.

L2 – Prediksi dulu, baru jalankan (12 poin)

Isi kolom **Prediksi** sebelum menjalankan program. Tulis baris-baris output sesuai urutan, misalnya `A1, A2, A4, A5`.

`FlowDemo` bagian A, B, C (7 poin: A 2, B 3, C 2)

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
A: `partA("12")` lalu `partA("12rb")`	A1, A2, A4, A5 A1, A3, A4, A5	== Bagian A == A1 mulai A2 jumlah = 12 A4 finally A5 lanjut A1 mulai A3 bukan angka A4 finally A5 lanjut	Cocok	Tebakan benar karena “12” memang hanya berisi angka, karena itulah input “12” menampilkan A2. Sedangkan “12rb” memiliki isi selain angka bulat (huruf) yang menyebabkan input tidak bisa di parse ke jenis int, jadi program langsung memanggil A3.

Bagian	Prediksi output	Output sebenarnya	Cocok?	Penjelasan selisih (atau mengapa tebakan Anda benar)
B: `partB(300000, "3")` lalu `partB(300000, "0")`	B1, B2, B4, B5 B1, B4, B7 main menangkap: / by zero	== Bagian B == B1 bulan = 3 B2 cicilan = 100000 B4 finally B5 hasil = 100000 B1 bulan = 0 B4 finally B7 main menangkap: / by zero	Cocok	Tebakan benar karena pada input pertama bulan tertulis “3”, sedangkan input kedua bulan tertulis “0”. Bulan “3” menghasilkan monthly yang didapat dari pembagian total dengan bulan. B5 menunjukkan hasil akhirnya. Sedangkan pada input bulan “0”, hasil tidak ditunjukkan dan langsung skip ke B4 dan B7. B7 pun menangkap pesan error dari pembagian tadi yang muncul akibat pembagian dengan 0. Masalahnya adalah catch yang terdapat di potongan kode program tidak cocok dengan kondisi gagal yang terdapat pada contoh.
C: `partC(5)`	C1, C2, C3 sisa = 4	== Bagian C == C1 cek stok C2 finally C3 sisa = 4	Cocok	Tebakan benar karena program menjalankan perintah C1 cek stok, kemudian menemukan return stock - 1 yang menghitung sisa stok menjadi 4. Namun, sebelum nilai tersebut dikembalikan ke pemanggil fungsi, blok finally tetap dijalankan sehingga menampilkan C2 finally. Setelah itu, nilai hasil return ditampilkan oleh C3 sisa = 4. Hal ini menunjukkan bahwa blok finally tetap dijalankan meskipun terdapat perintah return di dalam blok try.

`CariProdukDemo` bagian D (3 poin)

Perintah	Prediksi output	Output sebenarnya	Penjelasan: catch mana yang jalan, dan mengapa
`... CariProdukDemo 2`	D1, D5	D1 SKU: RDG-002 D5 pencarian selesai	Tidak ada catch yang jalan, karena input sesuai dengan ketentuan.
`... CariProdukDemo abc`	D2, D5	D2 nomor harus bilangan bulat D5 pencarian selesai	NumberFormatException, karena input yang diberikan bukan bilangan bulat

Perintah	Prediksi output	Output sebenarnya	Penjelasan: catch mana yang jalan, dan mengapa
`...CariProdukDemo 8`	D3, D5	D3 nomor di luar katalog: Index 7 out of bounds for length 3 D5 pencarian selesai	ArrayIndexOutOfBoundsException, karena pada katalog hanya ada 3 produk (yang karenanya diset index size katalognya menjadi 3), sedangkan input yang dimasukkan adalah produk 8 (melebihi index), yang mana tidak ada pada katalog.
`...CariProdukDemo` (tanpa argumen)	D4, D5	D4 Berikan satu nomor produk. D5 pencarian selesai	IllegalArgumentException, karena tidak ada input yang dimasukkan, dan error akan dipanggil jika Panjang argument yang dimasukkan bernilai 0 (alias tidak ada isinya sama sekali).

L2.c – Ubah urutan catch (2 poin). Tempel pesan error compiler setelah catch `IllegalArgumentException` dipindah ke paling atas, lalu jelaskan artinya dalam satu kalimat.

Pesan Error: Unreachable catch block for NumberFormatException. It is already handled by the catch block for IllegalArgumentException

Artinya: Catch block NumberFormatException tidak terpanggil karena error sudah ditangani (di-catch) oleh Catch blok IllegalArgumentException (karena letaknya berada di atas NumberFormatException).

L3 – Custom exception dan checkout (10 poin)

(a) Error berantai (2 poin). Setelah `PaymentMethod.pay` diubah menjadi `void ... throws PaymentFailedException`, berapa error yang muncul, dan di file mana saja? Mengapa satu perubahan di interface memengaruhi file-file itu?

Jumlah error: 11 Error File:
BankTransferPayment.java, CreditCardPayment.java, EWalletPayment.java, InstallmentPayment.java, RefundablePayment.java, dan CheckoutService.java.

Alasan:
Perubahan pada interface memengaruhi file-file tersebut karena class pembayaran mengimplementasikan PaymentMethod, sedangkan CheckoutService menggunakan method pay(). Oleh karena itu, implementasi dan pemanggilan method harus disesuaikan dengan kontrak baru.

(b) Enam skenario `ScenarioRunner` (4 poin). Isi kolom prediksi dulu (pakai parameter L0), baru jalankan. Hasil diisi salah satu dari `[OK]`, `[SALDO]`, `[BAYAR]`, `[STOK]`, `[INVALID]`.

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S1 Kopi + Rendang, Reguler, e-wallet	Berhasil	3 / 0 / 5 / Rp35.000	[OK]	stok KOPI=3 RDG=0 TEH=5 | saldo Rp35000	Cocok
S2 Kopi x3, Reguler, transfer bank	Gagal, Transfer bank gangguan	3 / 0 / 5 / Rp35.000	[BAYAR]	stok KOPI=3 RDG=0 TEH=5 | saldo Rp35000	Cocok

No.	Prediksi hasil	Prediksi sesudahnya: KOPI / RDG / TEH / saldo	Hasil sebenarnya	Sebenarnya: KOPI / RDG / TEH / saldo	Cocok?
S3 Rendang x2, Reguler, e-wallet	Gagal, stok tidak mencukupi	3 / 0 / 5 / Rp35.000	[STOK]	stok KOPI=3 RDG=0 TEH=5 | saldo Rp35000	Cocok
S4 Kopi, Ekspres, kartu kredit	Gagal, limit kartu kredit tidak cukup	3 / 0 / 5 / Rp35.000	[SALDO]	stok KOPI=3 RDG=0 TEH=5 | saldo Rp35000	Cocok
S5 Keranjang kosong	Gagal, tidak ada barang untuk checkout	3 / 0 / 5 / Rp35.000	[INVALID]	stok KOPI=3 RDG=0 TEH=5 | saldo Rp35000	Cocok
S6 Teh x2, Reguler, e-wallet	Gagal, saldo tidak cukup	3 / 0 / 5 / Rp35.000	[SALDO]	stok KOPI=3 RDG=0 TEH=5 | saldo Rp35000	Cocok

Pilih satu skenario yang prediksinya meleset (atau, jika semua cocok, skenario yang paling menarik), lalu jelaskan mengapa hasilnya seperti itu. Sebut juga mengapa pesan `[STOK]` menyebut "diminta 1".

Skenario S2 menarik karena meskipun stok kopi mencukupi untuk membeli 3 buah, transaksi tetap gagal karena Bank Nagari sedang offline (online = false). Sebelum pembayaran dilakukan, sistem melakukan reservasi stok sebanyak 3 kopi. Ketika pembayaran gagal, CheckoutService menjalankan rollback sehingga stok kopi dikembalikan ke jumlah semula.

Pesan [STOK] menyebut **"diminta 1"** karena proses reservasi dilakukan satu produk setiap pemanggilan. Jadi, meskipun keranjang berisi 3 kopi, sistem memeriksa dan mereservasi stok secara bertahap, masing-masing sebanyak 1 buah.

(c) Mengembalikan stok lalu `throw e` (2 poin). Di method `checkout`, mengapa blok catch mengembalikan stok lalu melempar ulang exception yang sama, bukan mencetak pesan untuk pembeli? Apa yang terjadi pada stok jika `release` dihapus?

Blok catch pada checkout bertugas menangani kegagalan transaksi dengan mengembalikan stok yang sebelumnya sudah direservasi (*rollback*), kemudian melempar ulang exception yang sama agar dapat ditangani oleh bagian pemanggil, yaitu buy(). Pesan untuk pembeli sebaiknya dicetak di buy() karena bagian tersebut bertanggung jawab menampilkan informasi transaksi. Jika release dihapus, stok yang sudah direservasi tidak akan dikembalikan ketika transaksi gagal sehingga jumlah stok menjadi berkurang secara keliru.

(d) Urutan catch di `buy` (2 poin). Mengapa catch `InsufficientBalanceException` harus ditulis di atas catch `PaymentFailedException`? Apa yang terjadi jika urutannya ditukar?

InsufficientBalanceException harus ditulis sebelum PaymentFailedException karena InsufficientBalanceException merupakan subclass dari PaymentFailedException. Jika urutannya ditukar, PaymentFailedException akan menangkap exception tersebut terlebih dahulu sehingga catch InsufficientBalanceException menjadi tidak dapat dijangkau (*unreachable code*) dan program mengalami compilation error.

L4 – Katalog dari file (6 poin)

(a) Baris katalog yang dilewati (3 poin).

Baris ke-	Isi baris	Apa masalahnya?	Exception yang tertangkap (nama class)
3	KUE-003;Kue Lapis Legit;95rb;800	Harga 95rb bukan angka sehingga gagal dikonversi menjadi long.	NumberFormatException
5	BATIK-005;Kain Batik Tulis	Jumlah kolom kurang dari 4, hanya ditemukan 2 kolom.	IllegalArgumentException
6	SAMB-006;Sambal Lado Mudo 200g;-45000;250	Harga bernilai negatif, sedangkan konstruktor PhysicalProduct tidak menerima harga negatif.	IllegalArgumentException

(b) Ketika file tidak ada (3 poin). Tempel output `java -cp bin shop.app.ShopApp katalog-cadangan.csv`. Apa isi `e.getMessage()` dan `e.getCause()`? Kalau `ShopApp` hanya mencetak `e.getCause().getMessage()`, apa yang dilihat pengguna, dan mengapa itu kurang membantu?

```
Toko tidak bisa dibuka: Katalog gagal dibaca: katalog-cadangan.csv
Penyebab asli: java.nio.file.NoSuchFileException: katalog-cadangan.csv
```

e.getMessage() berisi pesan dari CatalogLoadException, yaitu "Katalog gagal dibaca: katalog-cadangan.csv". Sementara itu, e.getCause() berisi exception penyebab aslinya, yaitu NoSuchFileException karena file katalog tidak ditemukan.

Jika ShopApp hanya mencetak e.getCause().getMessage(), pengguna hanya melihat pesan bahwa file katalog-cadangan.csv tidak ditemukan. Hal ini kurang membantu karena tidak menjelaskan konteks bahwa proses pembukaan toko gagal akibat katalog tidak dapat dibaca. Dengan membungkus exception, pengguna mendapatkan informasi error yang lebih jelas, sedangkan penyebab aslinya tetap bisa ditelusuri untuk debugging.

L5 – Code review FragileShop (10 poin)

Tabel temuan (8 poin, 2 per method).

Method	Apa masalahnya?	Dampaknya (bagi pembeli atau programmer)	Perbaikan
`payOrder`	PaymentFailedException ditangkap, tetapi blok catch dibiarkan kosong sehingga exception tertelan.	**Pembeli:** Mengira pembayaran berhasil, padahal transaksi gagal. **Programmer:** Sulit mengetahui kegagalan karena exception tidak ditampilkan atau dicatat.	Tampilkan pesan kegagalan atau lempar kembali exception agar bisa ditangani oleh pemanggil.

Method	Apa masalahnya?	Dampaknya (bagi pembeli atau programmer)	Perbaikan
`isInStock`	Menggunakan reserve() dan release() untuk memeriksa stok, padahal pemeriksaan stok seharusnya tidak mengubah data.	**Pembeli:** Bisa mendapat informasi stok yang tidak akurat jika proses reservasi dan pengembalian terganggu. **Programmer:** Berisiko menimbulkan efek samping pada data stok.	Tambahkan method seperti hasStock(sku, quantity) pada Inventory untuk memeriksa ketersediaan stok tanpa mengubah jumlahnya.
`readCatalogHeader`	Membuat CatalogLoadException dengan cause bernilai null, sehingga penyebab asli tidak ikut disimpan.	**Pembeli:** Hanya mendapat informasi umum bahwa katalog gagal dibaca. **Programmer:** Kesulitan mencari sumber masalah karena penyebab asli exception hilang.	Gunakan new CatalogLoadException("Katalog gagal dibaca: " + file, e) agar penyebab asli tetap tersimpan.
`addToCart`	Kesalahan input angka dan stok habis ditangani dalam satu blok catch dengan pesan teknis langsung dari exception.	**Pembeli:** Mendapat pesan error teknis yang membingungkan dan tidak bisa membedakan salah input dengan stok habis. **Programmer:** Sulit mengevaluasi jenis kesalahan karena dua exception ditangani dengan cara yang sama.	Pisahkan penanganan `NumberFormatException` dan `OutOfStockException`, lalu berikan pesan yang mudah dipahami pembeli.

Output `FragileShop` setelah `payOrder` dan `readCatalogHeader` diperbaiki (2 poin).

```
1) Pembayaran gagal: Saldo tidak cukup: perlu Rp85000, tersedia Rp50000
2) Kopi tersedia? true
3) Gagal: For input string: "dua"
   Gagal: Stok KOPI-001 tidak cukup: diminta 5, tersedia 1
4) Katalog gagal dibaca: katalog-lama.csv | penyebab: java.nio.file.NoSuchFileException: katalog-lama.csv
```

Penutup

Hasil SelfCheck terakhir. Salin dua baris berikut dari output SelfCheck.

```
=== HASIL: 24 / 24 pemeriksaan lulus (nilai SelfCheck 50 / 50) ===
```

```
=== TANTANGAN: Tidak ditampilkan pada output SelfCheck.
```

Refleksi (4 poin).

1. (2 poin) Mengapa kesalahan ketik pembeli (misalnya `dua` sebagai jumlah) perlu ditangani berbeda dari stok yang habis? Hubungkan dengan pilihan checked/unchecked.

Kesalahan ketik seperti memasukkan "dua" sebagai jumlah termasuk NumberFormatException yang bersifat unchecked karena terjadi saat proses konversi input. Sementara itu, stok yang habis ditangani menggunakan OutOfStockException yang bersifat checked karena merupakan kondisi bisnis yang perlu diantisipasi oleh program. Keduanya perlu ditangani secara terpisah agar pembeli mendapatkan pesan kesalahan yang sesuai.

2. (1 poin) Apakah pesan exception teknis, misalnya `For input string: "dua"`, sebaiknya ditampilkan langsung kepada pembeli? Beri satu contoh dari praktikum ini.

Tidak, karena pesan exception teknis dapat membingungkan pembeli. Contohnya, daripada menampilkan For input string: "dua", program sebaiknya menampilkan pesan yang lebih mudah dipahami seperti "Jumlah harus berupa angka."

3. (1 poin, dinilai penuh asal diisi jujur) Satu hal tentang exception yang masih membingungkan atau ingin Anda tanyakan di pertemuan berikutnya.

Saya masih perlu mempelajari kembali sebagian besar materi exception handling karena masih belum sepenuhnya memahami cara kerja dan penerapannya. Saya ingin memperdalam pemahaman tentang jenis-jenis exception, penggunaan try-catch, serta kapan sebaiknya menggunakan throw dan throws.